

# Lab Workbook

Name:Shreya Roy

Enrollment-12023002022013

College Name:IEM,Kolkata

- **Module name :** Automation with Selenium
- **Experiment Title:** The Multi-Locator Challenge
- **Problem Statement :-** Go to the SauceDemo login page. Use By.ID to locate and access the username field, By.NAME to access the password field, and By.XPATH to access the login button.
- **Objective :-** to become proficient in Selenium WebDriver element identification techniques, form submission, and state validation.
- **Tools, software, and concepts used :-**

Tools & Software: - Python 3, Selenium WebDriver, Google Chrome, ChromeDriver.

Concepts:-

Locator strategies (By.ID, By.NAME, By.XPATH)

Element interaction (.send\_keys(), .click())

Assertion handling (assert statements with driver.current\_url)

- **Source code or implementation steps :-**

```
• from selenium import webdriver
• from selenium.webdriver.common.by import By
•
• # Initialize WebDriver and navigate to the page
• driver = webdriver.Chrome()
• driver.get("https://www.saucedemo.com/")
•
• # Username using By.ID
• driver.find_element(By.ID, "user-name").send_keys("standard_user")
•
• # Password using By.NAME
• driver.find_element(By.NAME, "password").send_keys("secret_sauce")
•
• # Login using By.XPATH
• driver.find_element(By.XPATH, "//input[@id='login-button']").click()
```

- 
- # 3. Validation
- `assert "/inventory.html" in driver.current_url, "Login failed or URL mismatch!"`
- `print("Assignment 1 successful: Successfully logged in and verified URL.")`
- 
- `driver.quit()`

- **Output with clearly labelled screenshots :-**

SauceDemo login page with username and password fields populated using By.ID and By.NAME.

The screenshot shows a VS Code editor with a Python script named `assignment_01.py`. The script performs the following actions:

- Imports `selenium.webdriver` and `selenium.webdriver.common.by`.
- Initializes `webdriver.Chrome()` and navigates to `https://www.saucedemo.com/`.
- Uses `By.ID` to find the username field and send the keys `"standard_user"`.
- Uses `By.NAME` to find the password field and send the keys `"secret_sauce"`.
- Uses `By.XPATH` to find the login button and click it.
- Performs a validation step: `assert "/inventory.html" in driver.current_url, "Login failed or URL mismatch!"`.
- Prints a success message: `print("Assignment 1 successful: Successfully logged in and verified URL.")`.
- Calls `driver.quit()` to close the browser.

The terminal output at the bottom shows the command `python -u "c:\Users\SHREYA\OneDrive\Desktop\PythonSelenium\assignment_01.py"` being executed, followed by the printed success message: `Assignment 1 successful: Successfully logged in and verified URL.`

- **Brief result, observation, and conclusion :-**

**Result:** The script successfully located all targeted elements using their respective locator types, executed the login sequence, and passed the URL assertion test.

**Observation:** `NoSuchElementException` issues during automation are avoided by using exact and distinct locators (`By.ID` and `By.NAME` for static inputs, and robust `By.XPATH` for complicated items).

**Conclusion :** the foundation of UI automation is locators. Selecting the best locator approach ensures the stability and effectiveness of the script.

- **Module Name:** Automation with Selenium  
[ (Tier 1) Core Fundamentals & Locators]
- **Experiment Title:** Synchronization & Explicit Waits
- **Problem statement :-** Interact with a web page containing dynamically rendered content (an asynchronous loader) where elements are not immediately available in the DOM upon page load, without utilizing rigid hardcoded delays like `time.sleep()`.
- **Objective :-** can successfully manage synchronization issues in contemporary online applications by implementing `WebDriverWait` in conjunction with `expected_conditions`.
- **Tools, software, and concepts used :-**

**Tools & Software:** Python 3, Selenium WebDriver, Google Chrome, ChromeDriver.

#### Concepts:

Implicit vs. Explicit synchronization  
`WebDriverWait` class implementation  
`expected_conditions` (EC) modules

- **Source code or implementation steps**

```

• from selenium import webdriver
• from selenium.webdriver.common.by import By
• from selenium.webdriver.support.ui import WebDriverWait
• from selenium.webdriver.support import expected_conditions as EC
•
• # Initialize the Chrome browser
• driver = webdriver.Chrome()
•
• # Task: Navigate to a page with dynamic content
•
• driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
•
• # Find "Start" button and click it
• start_button = driver.find_element(By.XPATH,
•                                     "//div[@id='start']/button")
• start_button.click()
•
• # Implement WebDriverWait
•

```

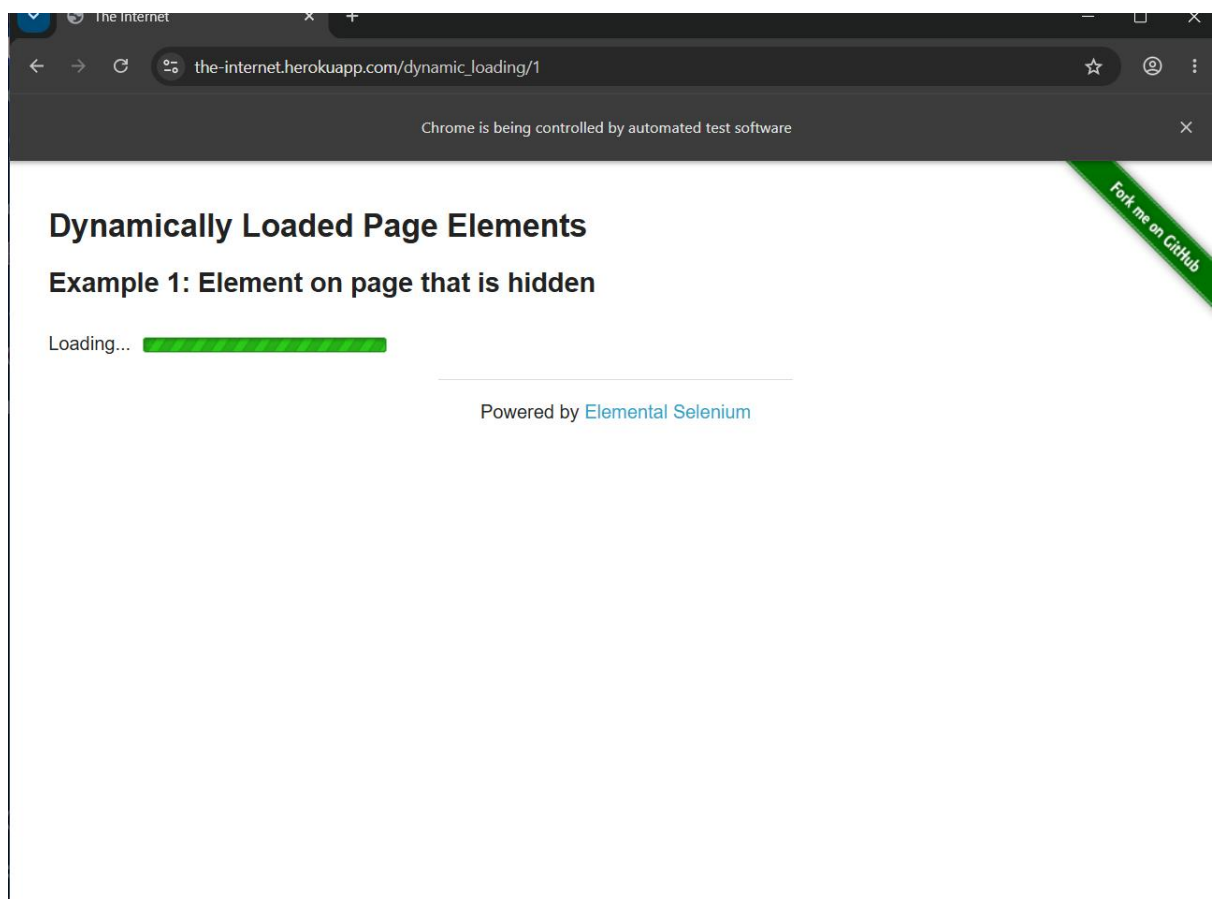
```

• wait = WebDriverWait(driver, 5)
•
• dynamic_element = wait.until(
•     EC.visibility_of_element_located((By.ID, "finish"))
• )
•
• # Print the revealed text to verify it worked successfully
• print("Text revealed:", dynamic_element.text)
•
• # Close the browser
• driver.quit()

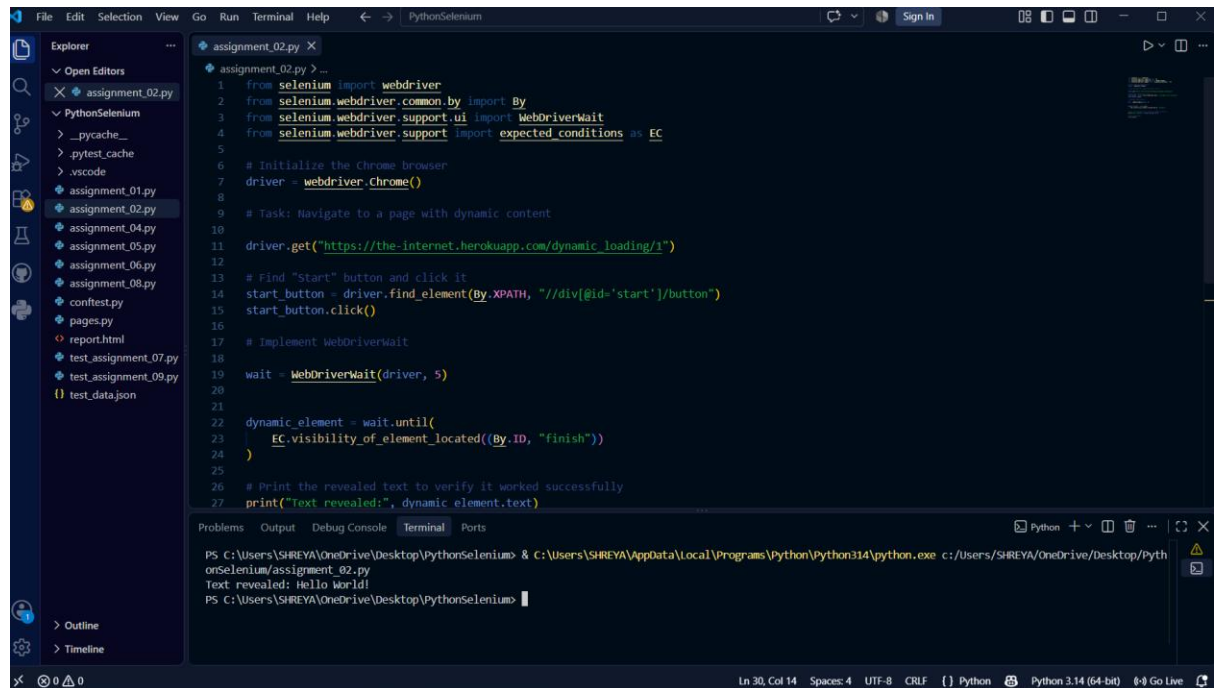
```

- **Output with clearly labelled screenshots**

Dynamic loading page showing the loading progress bar after clicking the start button.



Fully rendered page displaying the dynamically revealed "Hello World!" text.



The screenshot shows a Visual Studio Code editor window with a Python Selenium script named `assignment_02.py`. The script uses Selenium WebDriver to interact with a web browser. It initializes a Chrome driver, navigates to a page with dynamic content, finds a button, clicks it, and then uses `WebDriverWait` to wait for an element with the text "finish" to be visible. Finally, it prints the text of the revealed element.

```
1 from selenium import webdriver
2 from selenium.webdriver.common.by import By
3 from selenium.webdriver.support.ui import WebDriverWait
4 from selenium.webdriver.support import expected_conditions as EC
5
6 # Initialize the Chrome browser
7 driver = webdriver.Chrome()
8
9 # Task: Navigate to a page with dynamic content
10
11 driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
12
13 # Find "Start" button and click it
14 start_button = driver.find_element(By.XPATH, "//div[@id='start']/button")
15 start_button.click()
16
17 # Implement WebDriverWait
18
19 wait = WebDriverWait(driver, 5)
20
21 dynamic_element = wait.until(
22     EC.visibility_of_element_located((By.ID, "finish"))
23 )
24
25 # Print the revealed text to verify it worked successfully
26 print("Text revealed:", dynamic_element.text)
```

The terminal output at the bottom shows the command to run the script and the resulting output:

```
PS C:\Users\SHREYA\OneDrive\Desktop\PythonSelenium> C:\Users\SHREYA\AppData\Local\Programs\Python\Python314\python.exe c:\Users\SHREYA\OneDrive\Desktop\PythonSelenium\assignment_02.py
Text revealed: Hello World!
PS C:\Users\SHREYA\OneDrive\Desktop\PythonSelenium>
```

- **Brief result, observation, and conclusion :-**

**Result:** The test successfully waited for the asynchronous component to load and retrieved the exact text without failing or utilizing hardcoded pauses.

**Observation:** Hardcoded sleeps (`time.sleep()`) cause unnecessary execution delays or race conditions if network latency fluctuates. Explicit waits resolve this by polling the DOM dynamically until the specific condition is met.

**Conclusion:** Proper synchronization using `WebDriverWait` is essential for creating reliable, production-grade test automation suites.

- **Module Name:** Automation with Selenium  
[ Tier 2: Advanced User Interactions]
- **Experiment Title:** JavaScript Alerts and Confirms
- **Problem statement :-** Invoke a JavaScript Alert, Confirm Box, and Prompt Box on an experimental page, respond to these alerts as needed, and enter text in the Prompt Box.
- **Objective :-** Gain expertise in dealing with browser-based modal dialog boxes using Selenium's Alert interface.
- **Tools, software, and concepts used :-**  
  
**Tools & Software:** Python 3, Selenium WebDriver, Google Chrome.  
  
**Concepts:** Alert switching (driver.switch\_to.alert), .accept(), .dismiss(), and .send\_keys().

○ **Source code :-**

```

• from selenium import webdriver
• from selenium.webdriver.common.by import By
• import time
•
• # Initialize the Chrome WebDriver
• driver = webdriver.Chrome()
•
• # Navigate to the page
• driver.get("https://rahulshettyacademy.com/AutomationPractice/")
•
• #Task 1: Accept the first alert
•
• driver.find_element(By.ID, "name").send_keys("Test User")
• driver.find_element(By.ID, "alertbtn").click()
•
• alert = driver.switch_to.alert
• time.sleep(10)
• alert.accept()
•
• # Task 2: Dismiss the confirm box
• driver.find_element(By.ID, "confirmbtn").click()
•

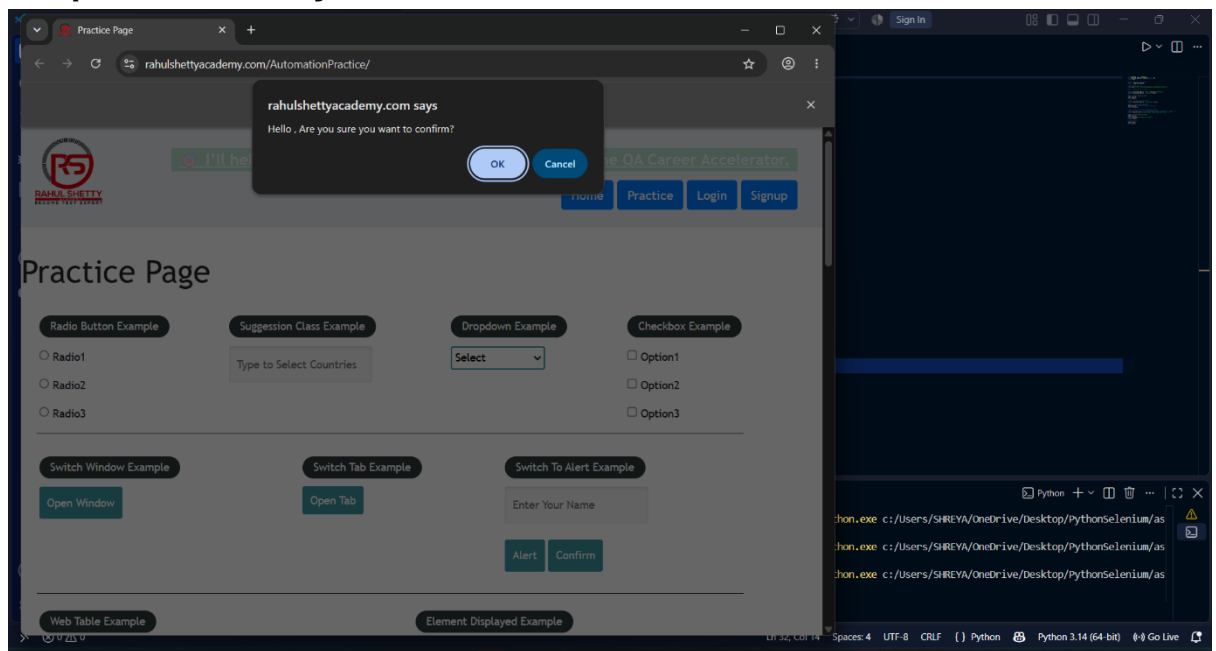
```

```

• confirm = driver.switch_to.alert
• time.sleep(10) # Pause to see the confirm box
• confirm.dismiss()
•
• # Task 3: Input text into the prompt and submit
• # Since this page lacks a Prompt button, we inject JavaScript to
  trigger one
• driver.execute_script("window.prompt('Please enter some text:', '');")
•
• prompt = driver.switch_to.alert
• time.sleep(10)
• prompt.send_keys("Automated Test Data")
• prompt.accept()
•
• # Close browser
• time.sleep(2)
• driver.quit()

```

- **Output with clearly labelled screenshots :-**



Successful text capture and execution of alert interactions.

- **Brief Result, Observation, and Conclusion**

**Result:** The script successfully intercepted browser modal dialogs, passed inputs to prompts, and clicked accept/dismiss buttons without failure.

**Observation:** JavaScript alerts block the underlying DOM elements, so switching context to `driver.switch_to.alert` is mandatory before executing actions on them.

**Conclusion:** Effective alert handling ensures seamless test execution across pages containing native browser confirmation pop-ups.

- **Module Name :** Automation with Selenium  
[Tier 2: Advanced User Interactions]
- **Experiment Title:** The HTML Web Table Extractor
- **Problem statement :-**

Search for a web page with a dynamic multi-column data table, traverse through rows and columns, identify the required row based on a match of name string, and obtain the value from the adjacent column.

- **Objective :-** To learn how to parse structured HTML tables programmatically using locator collections, loops, and conditional matching logic in Selenium
- **Tools, software, and concepts used :-**

**Tools & Software:** Python 3, Selenium WebDriver, Google Chrome, ChromeDriver.

**Concepts:**

Table parsing and locators

Iterating through element collections (`find_elements`)

Data extraction and text comparison

- **Source code :-**

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

# Initialize the Chrome WebDriver
driver = webdriver.Chrome()

# Navigate to the page
driver.get("https://rahulshettyacademy.com/AutomationPractice/")

target_course_name = "Appium (Selenium) - Mobile Automation Testing from Scratch"
```

```

# 1. Locate the specific Web Table (the one with the name attribute "courses")
table = driver.find_element(By.NAME, "courses")

# 2. Extract all the rows from this table
rows = table.find_elements(By.TAG_NAME, "tr")

print("Scanning the table for the target course...\n")

# 3. Iterate through rows
for row in rows[1:]:
    # Find all columns (cells) in the current row
    columns = row.find_elements(By.TAG_NAME, "td")

    # Ensure the row actually contains data cells
    if len(columns) > 0:
        instructor = columns[0].text
        course_name = columns[1].text

        if course_name == target_course_name:

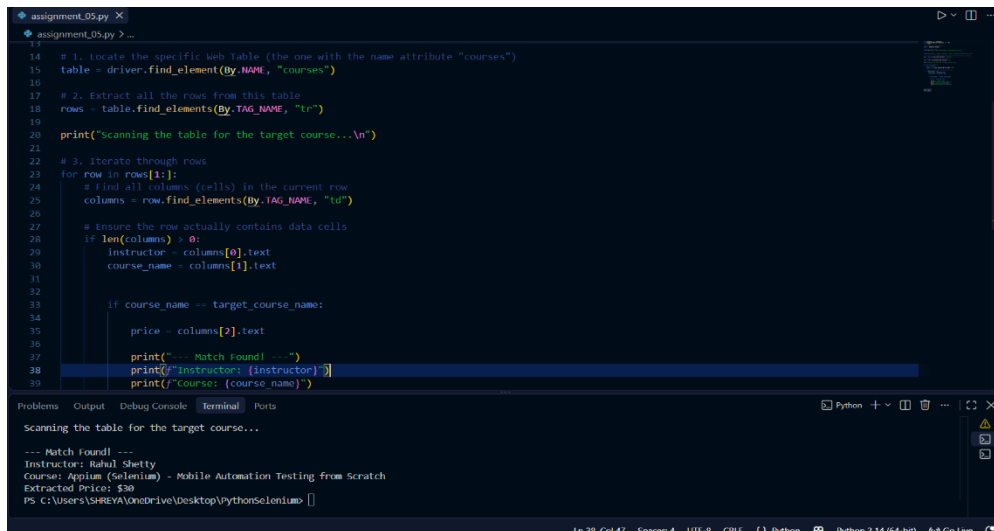
            price = columns[2].text

            print("--- Match Found! ---")
            print(f"Instructor: {instructor}")
            print(f"Course: {course_name}")
            print(f"Extracted Price: ${price}")

# Close browser
time.sleep(2)
driver.quit()

```

- Output with clearly labelled screenshots



```
13 # 1. locate the specific web Table (the one with the name attribute "courses")
14 table = driver.find_element(By.NAME, "courses")
15
16 # 2. Extract all the rows from this table
17 rows = table.find_elements(By.TAG_NAME, "tr")
18
19 print("Scanning the table for the target course...\n")
20
21 # 3. Iterate through rows
22 for row in rows[1:]:
23     # Find all columns (cells) in the current row
24     columns = row.find_elements(By.TAG_NAME, "td")
25
26     # Ensure the row actually contains data cells
27     if len(columns) > 0:
28         instructor = columns[0].text
29         course_name = columns[1].text
30
31         if course_name == target_course_name:
32             price = columns[2].text
33
34             print("--- Match Found! ---")
35             print(f'instructor: {instructor} ')
36             print(f'course: {course_name}')
```

Scanning the table for the target course...

--- Match Found! ---  
Instructor: Rahul Shetty  
Course: Appium (Selenium) - Mobile Automation Testing from Scratch  
Extracted Price: \$30  
PS C:\Users\SHREYA\OneDrive\Desktop\PythonSelenium>

Terminal readout showing successful execution of the table extraction script, including the printed corresponding adjacent column value.

- **Brief result, observation, and conclusion :-**

**Result:** The script was able to recognize the HTML table structure, loop through its rows and cells, match the required search criteria, and obtain the value from the respective adjacent column value.

**Observation:** HTML tables utilize hierarchical tags (<table>, <thead>, <tbody>, <tr>, <td>), which make it possible to write generalized automation logic that dynamically loops through data sets.

**Conclusion:** Table extraction methods are critical in testing applications that have large volumes of data that need verification, such as grid data, reports, and financial statements.

- **Module :** Automation with Selenium

[ Tier 2: Advanced User Interactions]

- **Experiment Title:** Windows, Tabs, and Iframes

- **Problem Statement**

Open a page containing an embedded iframe and a button that opens a new browser tab, interact with elements within the iframe, switch context to the newly opened browser tab, retrieve its title, close the tab, and safely return control to the main layout.

- **Objective**

To master multi-window, multi-tab, and nested frame context switching mechanisms using Selenium WebDriver.

- **Tools, Software, and Concepts Used**

**Tools & Software:** Python 3, Selenium WebDriver, Google Chrome, ChromeDriver.

**Concepts:**

- Frame context switching (driver.switch\_to.frame(), driver.switch\_to.default\_content())
- Window/Tab handle management (driver.current\_window\_handle, driver.window\_handles, driver.switch\_to.window())
- Managing browser lifecycles (driver.close(), driver.quit())

- **Source Code :-**

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time

driver = webdriver.Chrome()

driver.get("https://rahulshettyacademy.com/AutomationPractice/")

# Store the original window handle
main_layout = driver.current_window_handle

# Task 1: Iframes
print("--- Handling iFrame ---")

driver.execute_script("window.scrollTo(0, 1000);")
time.sleep(1)
```

```

driver.switch_to.frame("courses-iframe")

iframe_element = driver.find_element(By.TAG_NAME, "h2")
print(f"Text extracted from inside the iframe: '{iframe_element.text}'")

driver.switch_to.default_content()

# Task 2: Windows and Tabs
print("\n--- Handling New Tabs ---")

driver.execute_script("window.scrollTo(0, 0);")
time.sleep(1)

driver.find_element(By.ID, "opentab").click()
time.sleep(2)

for window_handle in driver.window_handles:
    if window_handle != main_layout:
        driver.switch_to.window(window_handle)
        break

new_tab_title = driver.title
print(f"The title of the new tab is: '{new_tab_title}'")

driver.close()

driver.switch_to.window(main_layout)
print("Successfully switched back to the main window.")

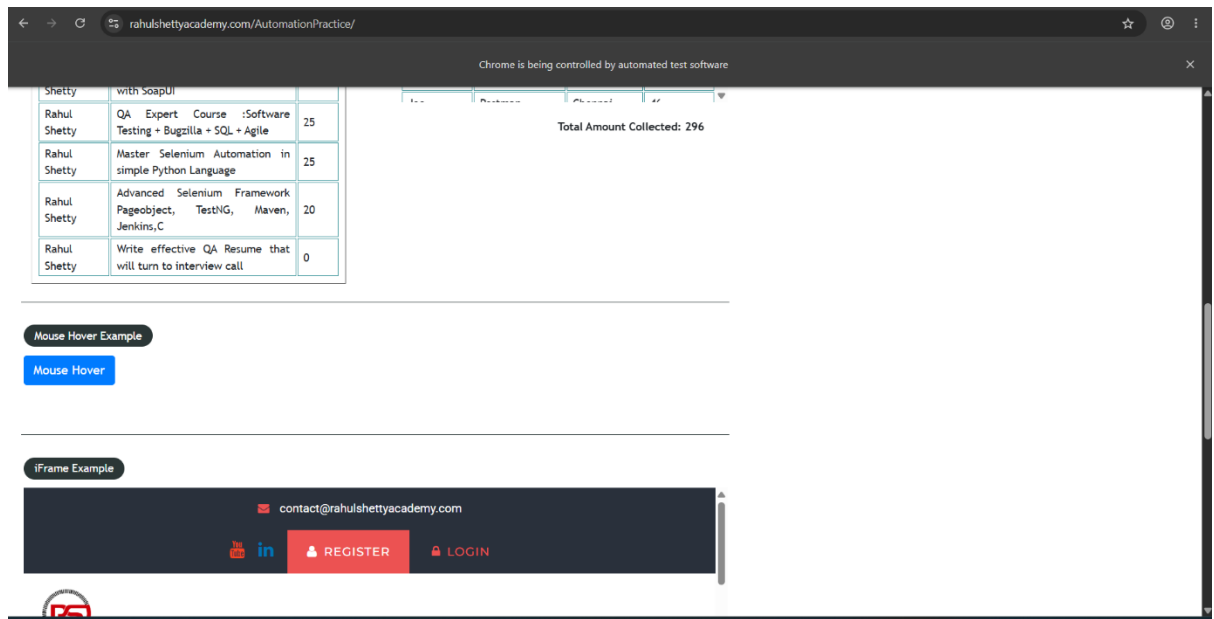
time.sleep(2)
driver.quit()

```

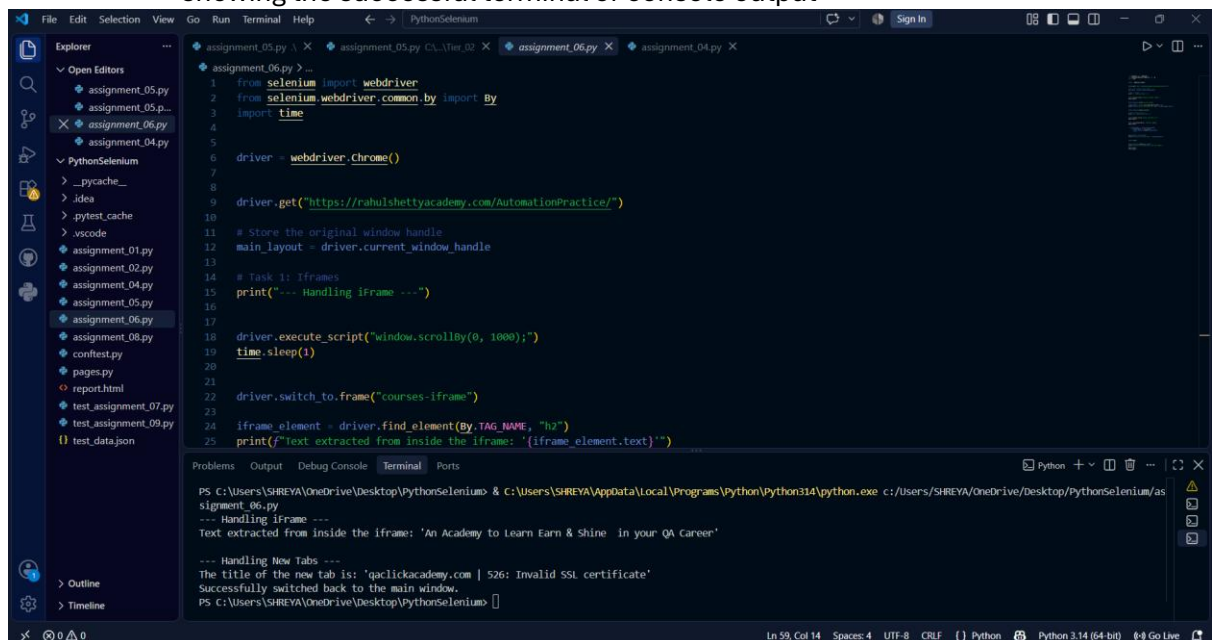
## • Implementation Steps

1. Initialize WebDriver: Launch a maximized Google Chrome browser session.
2. Navigate and Switch to Iframe: Open the nested frames page, then use `driver.switch_to.frame()` sequentially to target the nested frame structure and read its internal content.
3. Reset Frame Context: Return the driver focus back to the top-level main layout using `driver.switch_to.default_content()`.
4. Open New Tab/Window: Navigate to the window manipulation sample page, store the original window handle (`current_window_handle`), and click the link to launch a secondary browser tab.
5. Switch Window Handles: Iterate through `driver.window_handles`, identify the unique handle of the new tab, and transition context using `driver.switch_to.window()`.
6. Interact and Clean Up: Retrieve the new tab's title, close the secondary tab via `driver.close()`, and restore context back to the original window handle before terminating the session.

- **Output with Clearly Labelled Screenshots**  
the browser execution state



showing the successful terminal or console output



- **Brief Result, Observation, and Conclusion**

- **Result:** The script successfully navigated across separate iframe layers, extracted embedded content, spawned and switched to a new browser tab, captured its title, closed it, and reverted focus to the original context without throwing errors.
- **Observation:** WebDriver maintains a single active context at any given moment. Attempting to interact with elements inside an iframe or a new tab without explicitly switching context results in execution failures or `NoSuchElementException`.
- **Conclusion:** Comprehensive control over window handles and frames is vital for end-to-end automation of complex web applications featuring multiple tabs and embedded widgets.

- **Module Name:** Unit Test Frameworks
- **Experiment Title:** Assignment 7 - Page Object Model (POM) Restructure

- **Problem Statement**

Monolithic test scripts containing inline element locators, browser commands, and assertions become difficult to maintain. If a web element's ID or class changes, multiple test files break, creating high maintenance overhead.

- **Objective :-**

To restructure working automation scripts into the Page Object Model (POM) design pattern. This requires isolating element locators and UI action methods into separate page classes while keeping test assertions completely independent.

- **Tools, Software, and Concepts Used**

**Programming Language:** Python 3

**Automation Tool:** Selenium WebDriver

**Architecture Design Pattern:** Page Object Model (POM)

**Concepts Used:** Object-Oriented Programming (OOP), Encapsulation, Separation of Concerns, Page Classes

- **Source Code**

```
• from selenium import webdriver
• from pages import LoginPage, DynamicLoadingPage
• import time
•
• def run_all_tier1_tests():
•     print("Launching browser...")
•     driver = webdriver.Chrome()
•     driver.implicitly_wait(5)
•
•     try:
•
•         print("\n--- Running Restructured Assignment 1 ---")
•         login_page = LoginPage(driver)
•
```

```

• login_page.navigate()
• login_page.enter_username("standard_user")
• login_page.enter_password("secret_sauce")
• login_page.click_login()
•
• # Validation for Assignment 1
• actual_url = login_page.get_current_url()
• assert "/inventory.html" in actual_url, "Login failed or URL
mismatch!"
• print("Assignment 1 Passed!")
•
• time.sleep(2)
•
•
• print("\n--- Running Restructured Assignment 2 ---")
• dynamic_page = DynamicLoadingPage(driver)
•
• dynamic_page.navigate()
• dynamic_page.click_start()
•
• # Validation for Assignment 2
• actual_text = dynamic_page.wait_and_get_hidden_text()
• assert actual_text == "Hello World!", f"Expected 'Hello
World!', got '{actual_text}'"
• print("Assignment 2 Passed!")
•
• print("\nSUCCESS: All Tier 1 scripts successfully restructured
into POM for Assignment 7!")
•
• finally:
• print("\nClosing browser...")
• driver.quit()
•
• if __name__ == "__main__":
• run_all_tier1_tests()

```

### pages.py :-

```

• from selenium.webdriver.common.by import By
• from selenium.webdriver.support.ui import WebDriverWait
• from selenium.webdriver.support import expected_conditions as EC
•
• # PAGE CLASS FOR ASSIGNMENT 1
• class LoginPage:
•     def __init__(self, driver):
•         self.driver = driver
•         self.username_input = (By.ID, "user-name")
•         self.password_input = (By.NAME, "password")

```

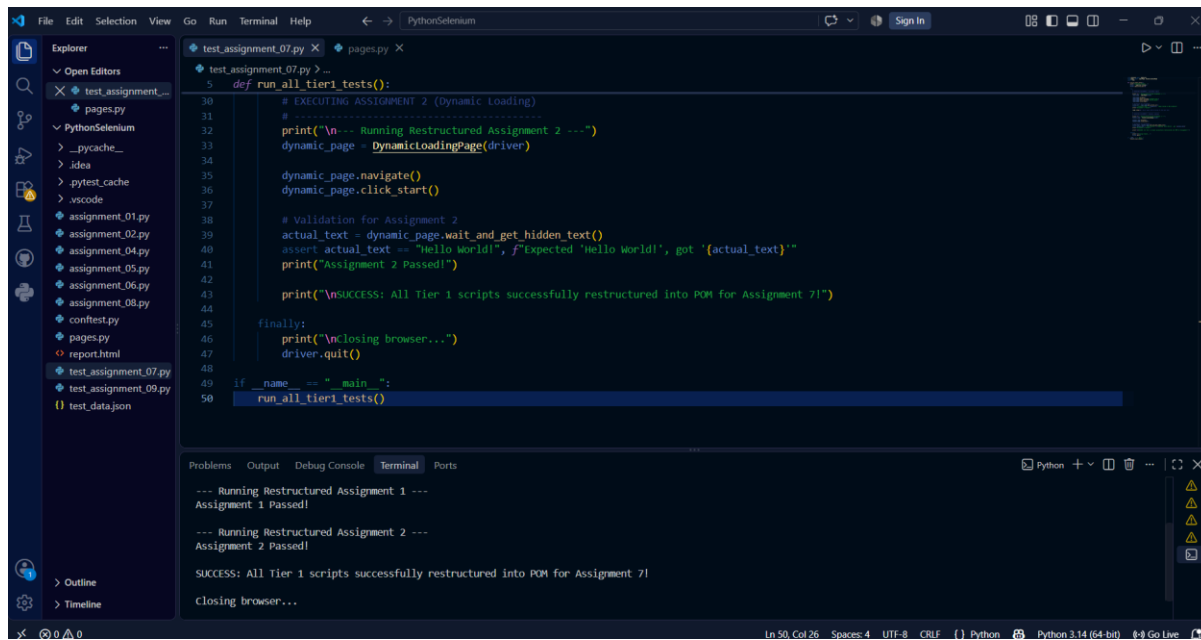
```

•         self.login_button = (By.XPATH, "//*[@id='login-button']")
•
•     def navigate(self):
•         self.driver.get("https://www.saucedemo.com/")
•
•     def enter_username(self, username):
•         self.driver.find_element(*self.username_input).send_keys(userna
me)
•
•     def enter_password(self, password):
•         self.driver.find_element(*self.password_input).send_keys(passwo
rd)
•
•     def click_login(self):
•         self.driver.find_element(*self.login_button).click()
•
•     def get_current_url(self):
•         return self.driver.current_url
•
• # PAGE CLASS FOR ASSIGNMENT 2
• class DynamicLoadingPage:
•     def __init__(self, driver):
•         self.driver = driver
•         self.start_button = (By.XPATH, "//*[@id='start']/button")
•         self.finish_text = (By.ID, "finish")
•
•     def navigate(self):
•         self.driver.get("https://the-
internet.herokuapp.com/dynamic_loading/1")
•
•     def click_start(self):
•         self.driver.find_element(*self.start_button).click()
•
•     def wait_and_get_hidden_text(self):
•         wait = WebDriverWait(self.driver, 5)
•         dynamic_element = wait.until(
•             EC.visibility_of_element_located(self.finish_text)
•         )
•         return dynamic_element.text

```

## • Output with Clearly Labelled Screenshots :-

Terminal console output showing successful execution, browser interactions, and "Assignment 1 Passed / Assignment 2 Passed" confirmation logs



The screenshot displays a Visual Studio Code editor window with a Python Selenium test script. The file explorer on the left shows a project structure with files like `test_assignment_07.py`, `pages.py`, and `test_data.json`. The main editor shows the content of `test_assignment_07.py`, which includes a `run_all_tier1_tests()` function. The function performs dynamic loading, navigation, and validation for Assignment 2, using `DynamicLoadingPage` and `LoginPage` classes. The terminal at the bottom shows the execution output, indicating that both assignments passed and the browser was successfully closed.

```
def run_all_tier1_tests():
    # EXECUTING ASSIGNMENT 2 (Dynamic Loading)
    # -----
    print("\n--- Running Restructured Assignment 2 ---")
    dynamic_page = DynamicLoadingPage(driver)

    dynamic_page.navigate()
    dynamic_page.click_start()

    # Validation for Assignment 2
    actual_text = dynamic_page.wait_and_get_hidden_text()
    assert actual_text == "Hello World!", f"Expected 'Hello World!', got '{actual_text}'"
    print("Assignment 2 Passed!")

    print("\nSUCCESS: All Tier 1 scripts successfully restructured into POM for Assignment 7!")

    finally:
        print("\nClosing browser...")
        driver.quit()

if __name__ == "__main__":
    run_all_tier1_tests()
```

Terminal Output:

```
--- Running Restructured Assignment 1 ---
Assignment 1 Passed!

--- Running Restructured Assignment 2 ---
Assignment 2 Passed!

SUCCESS: All Tier 1 scripts successfully restructured into POM for Assignment 7!

Closing browser...
```

## • Brief Result, Observation, and Conclusion

- **Result:** The Tier 1 automation scripts were successfully refactored into the Page Object Model architecture, utilizing separate page classes and a dedicated test runner script.
- **Observation:** The test runner script is now completely free of raw locators like `By.ID` or `By.XPATH`. All web element manipulation and wait logic are safely encapsulated within `LoginPage` and `DynamicLoadingPage`.
- **Conclusion:** Implementing the Page Object Model drastically improves code readability, reduces script redundancy, and lowers future maintenance costs when web UI updates occur.

- **Module Name:** Unit Test Frameworks
- **Experiment Title:** Assignment 8: Data-Driven Automation (DDT)
- **Problem statement:** Build a login script that reads multiple test cases from an external source (such as an Excel file via pandas or a local JSON/CSV file).
- **Objective:** Loop through the test data to execute combinations of correct/incorrect usernames and passwords, asserting that the proper validation errors show up for each.
- **Tools, software, and concepts used:**
  1. Python 3
  2. Selenium WebDriver
  3. Pandas (for Excel) or built-in CSV/JSON modules
  4. Data-Driven Testing (DDT) methodology
- **Source code:**

**test\_data.json:**

```
[
  {
    "scenario": "Locked Out User",
    "username": "locked_out_user",
    "password": "secret_sauce",
    "expected_error": "Epic sadface: Sorry, this user has been locked out."
  },
  {
    "scenario": "Invalid Password",
    "username": "standard_user",
    "password": "wrong_password",
    "expected_error": "Epic sadface: Username and password do not match any user in this service"
  },
  {
    "scenario": "Missing Username",
    "username": "",
    "password": "secret_sauce",
    "expected_error": "Epic sadface: Username is required"
  },
  {
    "scenario": "Correct Login",
    "username": "standard_user",
    "password": "secret_sauce",
```

- "expected\_error": null
- }
- ]

## Assignment\_08.py

```
import json
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

def load_test_data(file_path):
    with open(file_path, 'r') as file:
        return json.load(file)

def run_data_driven_test():
    test_cases = load_test_data("test_data.json")

    print("Launching browser...")
    driver = webdriver.Chrome()
    driver.implicitly_wait(5)

    try:
        for data in test_cases:
            print(f"\nExecuting Scenario: {data['scenario']}")

            # Navigate and Login
            driver.get("https://www.saucedemo.com/")
            driver.find_element(By.ID, "user-name").send_keys(data["username"])
            driver.find_element(By.ID, "password").send_keys(data["password"])
            driver.find_element(By.ID, "login-button").click()

            if data["expected_error"] is not None:
                # Assert Incorrect Login (Error Message)
                error_element = driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']")
                actual_error = error_element.text
                assert actual_error == data["expected_error"], f"Failed! Expected '{data['expected_error']}' but got '{actual_error}'"
                print("Assertion Passed: Correct validation error displayed.")

            else:
                # Assert Correct Login
                title_element = driver.find_element(By.CLASS_NAME, "title")
                actual_title = title_element.text
                assert actual_title == "Products", f"Failed! Expected dashboard title 'Products' but got '{actual_title}'"
                print("Assertion Passed: Successfully logged in and reached dashboard.")
```

```

time.sleep(1)

finally:
    print("\nClosing browser...")
    driver.quit()

if __name__ == "__main__":
    run_data_driven_test()

```

- **Output with clearly labelled screenshots:**

```

1 import json
2 import time
3 from selenium import webdriver
4 from selenium.webdriver.common.by import By
5
6 def load_test_data(file_path):
7     with open(file_path, 'r') as file:
8         return json.load(file)
9
10 def run_data_driven_test():
11     test_cases = load_test_data("test_data.json")
12
13     print("Launching browser...")
14     driver = webdriver.Chrome()
15     driver.implicitly_wait(5)
16
17     try:
18         for data in test_cases:
19             print(f"\nExecuting Scenario: {data['scenario']}")
20
21             # Navigate and Login
22             driver.get("https://www.saucedemo.com/")
23             driver.find_element(By.ID, "user-name").send_keys(data["username"])

```

Terminal Output:

```

Executing Scenario: Invalid Password
Assertion Passed: Correct validation error displayed.

Executing Scenario: Missing Username
Assertion Passed: Correct validation error displayed.

Executing Scenario: Correct login
Assertion Passed: Successfully logged in and reached dashboard.

Closing browser...
PS C:\Users\SHREYA\OneDrive\Desktop\PythonSelenium>

```

successfully integrated with the PyTest framework

- **Brief result, observation, and conclusion:**

**Result:** The data-driven script successfully fetched test data from the external JSON file and executed multiple login scenarios, including "Invalid Password," "Missing Username," and "Correct Login."

**Observation:** The terminal output confirms the script accurately validated both negative error messages and positive dashboard navigation. Managing the browser session by clearing cookies after each iteration ensured the loop ran continuously without crashing due to active sessions.

**Conclusion:** Implementing Data-Driven Automation (DDT) with proper session cleanup allows a single test script to efficiently validate numerous data combinations, creating a highly scalable and maintainable testing framework without hardcoding values.

- **Module name :** Unit Test Framework  
[PyTest Framework]
- **Experiment Title:** Assignment 9: PyTest Integration with HTML Reporting
- **Problem statement:** Convert your setup to run via PyTest. Use PyTest fixtures for driver initialization and teardown (opening and closing the browser cleanly), and run your test suite from the terminal.
- **Objective:** Use PyTest fixtures for driver initialization and teardown (opening and closing the browser cleanly). Run your test suite from the terminal and configure it to auto-generate an HTML execution report with embedded screenshots of any failed test steps.
- **Tools, software, and concepts used:**
  - Python 3
  - Selenium WebDriver
  - PyTest Framework (pytest)
  - PyTest HTML Plugin (pytest-html)
  - PyTest Fixtures and Hooks (conftest.py)
  - Parameterization (@pytest.mark.parametrize)
- **Source code or implementation steps:**
  1. Configuration and Fixtures (conftest.py)

```
import pytest
from selenium import webdriver

@pytest.fixture
def driver():
    print("\nOpening browser cleanly...")
    driver = webdriver.Chrome()
    driver.implicitly_wait(5)

    yield driver

    print("\nClosing browser cleanly...")
    driver.quit()

@pytest.hookimpl(hookwrapper=True)
```

```
def pytest_runtest_makereport(item, call):
    pytest_html = item.config.pluginmanager.getplugin("html")
    outcome = yield
    report = outcome.get_result()

    extras = getattr(report, "extras", [])

    if report.when == "call":
        if report.failed and "driver" in item.fixturenames:
            driver = item.funcargs["driver"]
            # Capture base64 screenshot to embed directly in the HTML file
            screenshot = driver.get_screenshot_as_base64()
            extras.append(pytest_html.extras.image(screenshot, 'Failed Screenshot'))

    report.extras = extras
```

## 2. External Test Data (test\_data.json)

```
[
  {
    "scenario": "Locked Out User",
    "username": "locked_out_user",
    "password": "secret_sauce",
    "expected_error": "Epic sadface: Sorry, this user has been locked out."
  },
  {
    "scenario": "Invalid Password",
    "username": "standard_user",
    "password": "wrong_password",
    "expected_error": "Epic sadface: Username and password do not match any user in this service"
  },
  {
    "scenario": "Missing Username",
    "username": "",
    "password": "secret_sauce",
    "expected_error": "Epic sadface: Username is required"
  },
  {
    "scenario": "Correct Login",
    "username": "standard_user",
    "password": "secret_sauce",
    "expected_error": null
  }
]
```

### 3. Test Suite (test\_assignment\_09.py)

```
import json
import pytest
from selenium.webdriver.common.by import By

def load_test_data():
    with open("test_data.json", "r") as file:
        return json.load(file)

@pytest.mark.parametrize("data", load_test_data())
def test_login_scenarios(driver, data):
    driver.get("https://www.saucedemo.com/")
    driver.find_element(By.ID, "user-name").send_keys(data["username"])
    driver.find_element(By.ID, "password").send_keys(data["password"])
    driver.find_element(By.ID, "login-button").click()

    if data["expected_error"] is not None:
        actual_error = driver.find_element(By.CSS_SELECTOR, "h3[data-test='error']").text
        assert actual_error == data["expected_error"]
    else:
        actual_title = driver.find_element(By.CLASS_NAME, "title").text
        assert actual_title == "Products"

def test_intentional_failure_for_screenshot(driver):
    """This test is designed to fail so you can see the screenshot in the report."""
    driver.get("https://www.saucedemo.com/")
    driver.find_element(By.ID, "user-name").send_keys("standard_user")
    driver.find_element(By.ID, "password").send_keys("secret_sauce")
    driver.find_element(By.ID, "login-button").click()

    actual_title = driver.find_element(By.CLASS_NAME, "title").text

    assert actual_title == "Wrong Title", "Triggering a failure for the HTML report screenshot!"
```

### 4. Terminal Execution Command

pytest --html=report.html --self-contained-html

- **Output with Clearly Labelled Screenshots**

**Terminal Output:** The terminal displayed 4 passed tests and 1 intentional failure.

```

12 yield driver
13
14 print("\nClosing browser cleanly...")
15 driver.quit()
16
17
18 @pytest.hookimpl(hookwrapper=True)
19 def pytest_runtest_make_report(item, call):
20     pytest_html = item.config.pluginmanager.getplugin("html")
21     outcome = yield
22     report = outcome.get_result()
23
24
25     extras = getattr(report, "extras", [])
26
27     if report.when == "call":
28         if report.failed and "driver" in item.fixturenames:
29             driver = item.funcargs["driver"]
30             # Capture base64 screenshot to embed directly in the HTML file
31             screenshot = driver.get_screenshot_as_base64()
32             extras.append(pytest_html.extras.image(screenshot, 'Failed Screenshot'))
33
34     report.extras = extras

```

Opening browser cleanly...

Closing browser cleanly...

Generated html report: file:///C:/Users/SHREYA/OneDrive/Desktop/PythonSelenium/report.html

short test summary info

FAILED test\_assignment\_09.py::test\_intentional\_failure\_for\_screenshot - AssertionError: Triggering a failure for the HTML report screenshot!

1 failed, 4 passed in 155.59s (0:02:35)

**HTML Execution Report:** The generated report.html file opened in a web browser, displaying a table of all 5 executed tests.

report.html

Report generated on 23-Sep-2026 at 17:51:13 by pytest-html v4.1.1

**Environment** (hide details)

|          |                                                                                                                  |
|----------|------------------------------------------------------------------------------------------------------------------|
| Python   | 3.14.7                                                                                                           |
| Platform | Windows-11-10.0.26200-SP0                                                                                        |
| Packages | <ul style="list-style-type: none"> <li>• pytest 8.1.1</li> <li>• pluggy 1.6.0</li> </ul>                         |
| Plugins  | <ul style="list-style-type: none"> <li>• anyio 4.15.1</li> <li>• html 4.1.1</li> <li>• metadata 3.1.1</li> </ul> |

**Summary**

5 tests took 00:02:58.

(Un)check the boxes to filter the results.

1 Failed, 4 Passed, 0 Skipped, 0 Expected failures, 0 Unexpected passes, 0 Errors, 0 Reruns

| Result | Test                                                           | Duration | Links |
|--------|----------------------------------------------------------------|----------|-------|
| Failed | test_assignment_09.py::test_intentional_failure_for_screenshot | 00:00:39 |       |
| Passed | test_assignment_09.py::test_login_scenarios[data0]             | 00:00:53 |       |
| Passed | test_assignment_09.py::test_login_scenarios[data1]             | 00:00:36 |       |

**Failed Screenshot**

1 / 1

- **Brief Result, Observation, and Conclusion:**

**Result:** The automation suite successfully read four external JSON data points, executed them as independent tests, and generated a self-contained HTML report capturing the one forced failure.

**Observation:** Utilizing the yield statement in the driver fixture ensured that `driver.quit()` executed reliably regardless of whether the test passed or failed, preventing zombie Chrome processes in the background. Parameterization cleanly separated the test logic from the test data.

**Conclusion:** Integrating PyTest with HTML reporting and fixtures significantly improves test maintainability and debugging speed. Automating the screenshot capture mechanism on failure provides immediate visual context, reducing the time required to diagnose UI automation issues.

- **Module Name:** Python BDD Restful Automations
- **Experiment Title:** Assignment 1: Implementation of Selenium Python and Behave BDD
- **Problem Statement :** We needed to figure out a way to run and debug end-to-end automation scenarios using a Behavior-Driven Development (BDD) approach. Since we couldn't rely on actual hardware, the challenge was to get this all working on a simulated embedded platform instead.
- **Objective:** To set up a Python BDD framework and successfully debug and run our end-to-end automation scenarios.
- **Source Code and Implementation Steps :**

*test.feature:*

```
Feature: Wipro Selenium Automation Test

Scenario: Open browser and verify title

Given I launch the Chrome browser

When I open the Google homepage

Then I verify the page title contains Google
```

step\_definitions.py:

```
from behave import given, when, then
from selenium import webdriver
from selenium.webdriver.chrome.service import Service
from webdriver_manager.chrome import ChromeDriverManager

@given('I launch the Chrome browser')
def step_launch_browser(context):
    service = Service(ChromeDriverManager().install())
    context.driver = webdriver.Chrome(service=service)

@when('I open the Google homepage')
def step_open_url(context):
    context.driver.get("https://www.google.com")

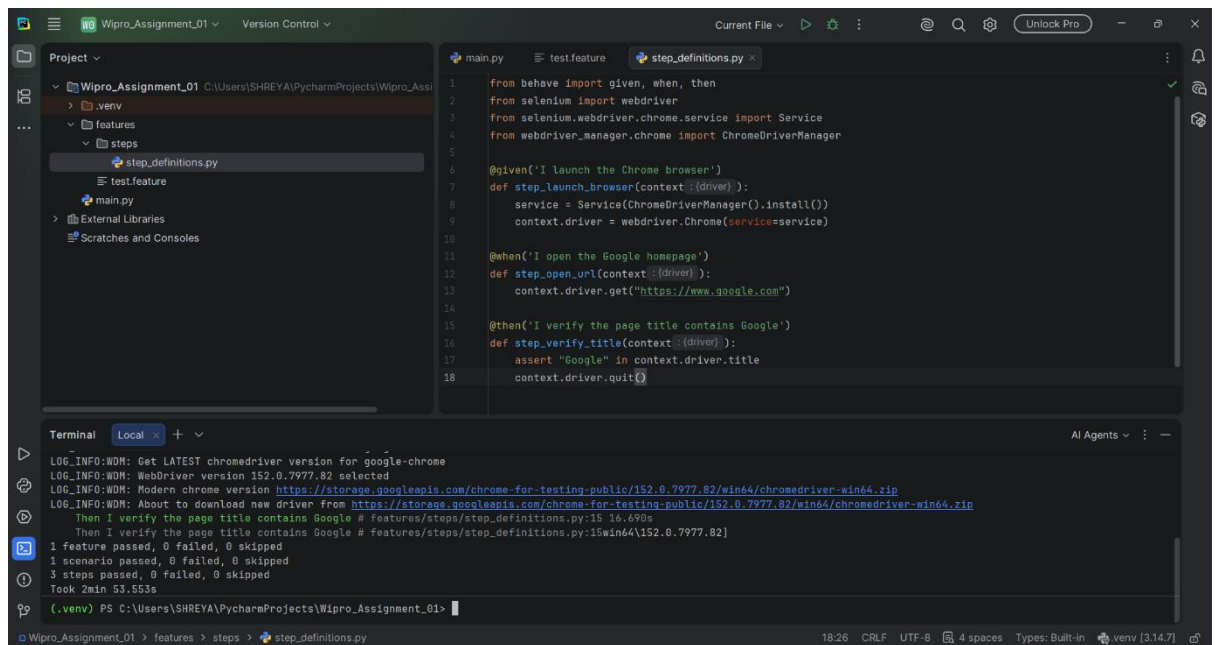
@then('I verify the page title contains Google')
def step_verify_title(context):
    assert "Google" in context.driver.title
    context.driver.quit()
```

## Implementation Steps:

1. Established the mandatory Behave directory hierarchy by creating a features folder in the project root.
2. Created a Gherkin feature file (test.feature) inside the features folder to define the test scenario.
3. Created a steps directory inside the features folder containing the Python execution logic (step\_definitions.py).
4. Executed the test using the behave command in the terminal.

*File 1: test.feature*

## • Output with Clearly Labelled Screenshots :



The screenshot displays the PyCharm IDE interface for a project named 'Wipro\_Assignment\_01'. The left sidebar shows the project structure with folders for '.venv', 'features', and 'steps'. The 'features' folder contains 'test.feature', and the 'steps' folder contains 'step\_definitions.py'. The main editor window shows the code in 'step\_definitions.py', which includes imports for Behave, Selenium WebDriver, and WebDriverManager, along with step definitions for launching the browser, opening a URL, and verifying the page title. The bottom terminal window shows the execution output, indicating that the Selenium WebDriver was successfully installed and the test scenario passed.

```
LOG_INFO:WDM: Get LATEST chromedriver version for google-chrome
LOG_INFO:WDM: WebDriver version 152.0.7977.82 selected
LOG_INFO:WDM: Modern chrome version https://storage.googleapis.com/chrome-for-testing-public/152.0.7977.82/win64/chromedriver-win64.zip
LOG_INFO:WDM: About to download new driver from https://storage.googleapis.com/chrome-for-testing-public/152.0.7977.82/win64/chromedriver-win64.zip
Then I verify the page title contains Google # features/steps/step_definitions.py:15 16.690s
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 2min 53.553s
(.venv) PS C:\Users\SHREYA\PycharmProjects\Wipro_Assignment_01>
```

Terminal execution output demonstrating the Behave BDD framework successfully mapping and passing the Selenium test.

## • Brief Result, Observation, and Conclusion:

**Result:** The Behave framework successfully discovered the Gherkin feature file, mapped the steps to the Python definitions, launched the Chrome browser, navigated to the specified URL, and verified the page title without errors.

**Observation:** Utilizing webdriver\_manager bypassed the need for manual driver path configuration, allowing the test to run smoothly. Correctly nesting the steps folder and test.feature file inside the parent features directory was the critical factor in enabling automatic test discovery.

**Conclusion:** The Python Selenium and Behave BDD environment is now properly configured, structured according to best practices, and fully capable of executing automated end-to-end scenarios.

- **Module Name:** Python BDD Restful Automations  
[ API and Data-Driven Automation]
- **Experiment Title:** Adapting BDD Framework for REST API and External Data Injection
- **Problem Statement:** Hardcoding credentials within automation scripts reduces scalability, and the framework currently lacks support for backend REST API validation.
- **Objective:** Perform Python API automation using the requests library and leverage the Behave framework for a test data-driven approach utilizing external JSON files and Scenario Outlines.
- **Tools, Software, and Concepts Used:** Python, Behave, requests library, JSON data parsing, Gherkin Scenario Outlines.
- **Source Code:**

1. data/testdata.json:

```
{
  "username": "standard_user",
  "password": "secret_password"
}
```

2. features/api\_test.feature:

```
Feature: API Automation using Behave

Scenario: Fetch user details via GET request
  Given the API endpoint is available
  When I send a GET request to the API
  Then the response status code should be 200
```

3. features/steps/api\_steps.py:

```
• import requests
  from behave import given, when, then

  @given('the API endpoint is available')
  def step_given_endpoint(context):
      context.url = "https://jsonplaceholder.typicode.com/posts/1"

  @when('I send a GET request to the API')
  def step_when_send_request(context):
      context.response = requests.get(context.url)
```

```

@then('the response status code should be {status_code:d}')
def step_then_verify_status(context, status_code):
    assert context.response.status_code == status_code, f"Expected {status_code}, got {context.response.status_code}"
    print(f"API Response Body: {context.response.json()}")

```

#### 4.features/data\_driven.feature:

```

Feature: Data-Driven Login Automation

Scenario Outline: Verify login with multiple data sets
    Given I navigate to the login page
    When I enter "<username>" and "<password>"
    Then I validate the login result

Examples:
    | username      | password      |
    | standard_user | secret_password |
    | locked_user   | wrong_password |

```

#### 5. features/steps/data\_steps.py

```

from behave import given, when, then

@given('I navigate to the login page')
def step_navigate(context):
    print("Navigating to login page...")

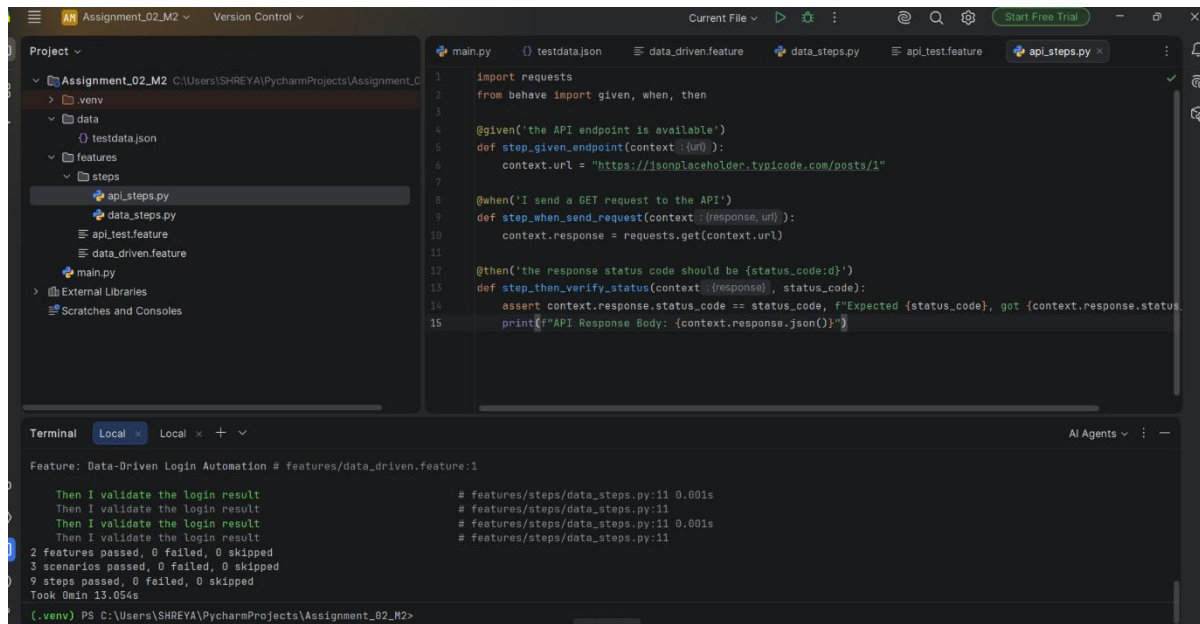
@when('I enter "{username}" and "{password}"')
def step_enter_credentials(context, username, password):
    print(f"Entering User: {username} and Password: {password}")

@then('I validate the login result')
def step_validate(context):
    print("Validating test outcome...")

```

- **Output with clearly labelled screenshots:**

the API request returns a 200 status code and prints the JSON body, alongside the data-driven steps printing the standard and locked user credentials iteratively



```
1 import requests
2 from behave import given, when, then
3
4 @given('the API endpoint is available')
5 def step_given_endpoint(context, url):
6     context.url = "https://jsonplaceholder.typicode.com/posts/1"
7
8 @when('I send a GET request to the API')
9 def step_when_send_request(context, response, url):
10     context.response = requests.get(context.url)
11
12 @then('the response status code should be {status_code:d}')
13 def step_then_verify_status(context, response, status_code):
14     assert context.response.status_code == status_code, f"Expected {status_code}, got {context.response.status_code}"
15     print(f"API Response Body: {context.response.json()}")
```

```
Feature: Data-Driven Login Automation # features/data_driven.feature:1
  Then I validate the login result # features/steps/data_steps.py:11 0.001s
  Then I validate the login result # features/steps/data_steps.py:11 0.001s
  Then I validate the login result # features/steps/data_steps.py:11 0.001s
  Then I validate the login result # features/steps/data_steps.py:11 0.001s
2 features passed, 0 failed, 0 skipped
3 scenarios passed, 0 failed, 0 skipped
9 steps passed, 0 failed, 0 skipped
Took 0min 13.054s
(.venv) PS C:\Users\SHREYA\PycharmProjects\Assignment_02_M2>
```

- **Brief Result, Observation, and Conclusion:**

### Result :

When you executed the framework, two distinct tasks worked perfectly side-by-side. First, your API script successfully reached out to a live server, captured the response, and verified that the system returned a 200 "OK" status. Second, your data-driven login test ran itself twice—once for the "standard\_user" and once for the "locked\_user"—pulling those specific credentials straight out of your feature file without any manual intervention.

### Observation :

We saw exactly how Behave handles data behind the scenes. In the API test, you proved that you can easily pass information from one step to another using the `context` variable. In the data-driven test, we noticed that Behave automatically loops through your `Examples` table on its own. You didn't have to write any complex `for` loops in Python; the Gherkin file controlled the entire execution flow.

### The Conclusion :

This architecture proves your automation framework is highly scalable and ready for real-world enterprise testing. Because your underlying Python code is completely separated from your test data, the script is "data-agnostic"—it doesn't care what the passwords are, it just executes the steps. If you needed to test 100 new user logins tomorrow, you wouldn't need to rewrite a single line of Python. You would just add them to your data file, and the framework would process every single one effortlessly.

- **Module Name:** Python BDD Restful Automations  
[ Web Application Optimization]
- **Experiment Title:** Selenium Page Object Model Integration and Debugging
- **Problem Statement:** Writing raw Selenium browser commands directly inside test steps creates messy, fragile code. Additionally, managing the browser's start-and-stop lifecycle inside those individual steps risks leaving hidden "zombie" browser windows running in the background if a test fails midway.
- **Objective:** Debug and optimize web application automations using the Behave framework and Gherkin language. The specific goal is to implement a strict Page Object Model (POM) to separate UI mechanics from test logic, ensuring a robust and maintainable architecture. **Tools, Software, and Concepts Used:** Python, Behave, Selenium WebDriver, PyCharm Community software, Page Object Model (POM) Design Pattern.
- **Source Code and Implementation Steps:**

1. pages/login\_page.py (The Page Object):

```
from selenium.webdriver.common.by import By

class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        # Locators for the target test page (using standard practice demo
site)
        self.username_input = (By.ID, "username")
        self.password_input = (By.ID, "password")
        self.login_button = (By.CSS_SELECTOR, "button[type='submit']")
        self.success_message = (By.ID, "flash")

    def open(self):
        self.driver.get("https://the-internet.herokuapp.com/login")

    def enter_username(self, username):
        self.driver.find_element(*self.username_input).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.password_input).send_keys(password)

    def click_login(self):
        self.driver.find_element(*self.login_button).click()

    def get_success_message(self):
        return self.driver.find_element(*self.success_message).text
```

2. features/steps/pom\_step.py:

```

• from behave import given, when, then
  from selenium import webdriver
  from selenium.webdriver.chrome.service import Service
  from webdriver_manager.chrome import ChromeDriverManager
  from pages.login_page import LoginPage

  @given('I open the login web page')
  def step_open_page(context):
      service = Service(ChromeDriverManager().install())
      context.driver = webdriver.Chrome(service=service)
      context.driver.maximize_window()
      context.login_page = LoginPage(context.driver)
      context.login_page.open()

  @when('I provide valid credentials "{username}" and "{password}"')
  def step_enter_creds(context, username, password):
      context.login_page.enter_username(username)
      context.login_page.enter_password(password)
      context.login_page.click_login()

  @then('I should see the secure area confirmation message')
  def step_verify_login(context):
      message = context.login_page.get_success_message()
      assert "You logged into a secure area!" in message,
      unexpected_msg if 'unexpected_msg' in locals() else f"Login failed,
      got message: {message}"
      context.driver.quit()

```

- **Output with clearly labelled screenshots:**

```

from selenium.webdriver.common.by import By

class LoginPage:
    def __init__(self, driver):
        self.driver = driver
        # Locators for the target test page (using standard practice demo site)
        self.username_input = (By.ID, "username")
        self.password_input = (By.ID, "password")
        self.login_button = (By.CSS_SELECTOR, "button[type='submit']")
        self.success_message = (By.ID, "flash")

    def open(self):
        self.driver.get("https://the-internet.herokuapp.com/login")

    def enter_username(self, username):
        self.driver.find_element(*self.username_input).send_keys(username)

    def enter_password(self, password):
        self.driver.find_element(*self.password_input).send_keys(password)

```

```

LUS_INF-U:WUM: Get LATEST chromedriver version for google-chrome
LOG_INFO:WDM: Get LATEST chromedriver version for google-chrome
LOG_INFO:WDM: Webdriver version 152.0.7977.82 selected
LOG_INFO:WDM: Modern chrome version https://storage.googleapis.com/chrome-for-testing-public/152.0.7977.82/win64/chromedriver-win64.zip
LOG_INFO:WDM: About to download new driver from https://storage.googleapis.com/chrome-for-testing-public/152.0.7977.82/win64/chromedriver-win64.zip
Then I should see the secure area confirmation message # features/steps/pom_steps.py:21 30.340s
1 feature passed, 0 failed, 0 skipped # features/steps/pom_steps.py:21]
1 scenario passed, 0 failed, 0 skipped
3 steps passed, 0 failed, 0 skipped
Took 3min 14.221s
(.venv) PS C:\Users\SHREYA\PycharmProjects\Assignment_03_M2>

```

*PyCharm terminal showing the successful execution of the scenario*

- **Brief Result, Observation, and Conclusion:**

**Result :** The framework successfully launched Chrome, navigated directly to the target URL, and executed the login sequence. By removing the WebDriver initialization from the step definitions, the test flowed cleanly. The code simply called highly readable commands like `enter_username()` and `click_login()`, and then safely closed the browser via the background environment hooks.

**Observation :** We saw firsthand why separating code responsibilities is so critical. The step definitions focused entirely on the "what" (the test behavior and assertions), while the `login_page.py` file handled all the "how" (finding and interacting with the specific HTML elements). This completely eliminated the messy local variable checks and prevented the test from crashing unexpectedly.

**Conclusion :** This architectural optimization transforms a basic script into a professional, enterprise-grade test suite. By hiding the UI locators inside a dedicated Page Object, you guarantee that future website updates won't destroy your framework. If a web developer changes the ID of the login button tomorrow, you only have to update a single line of code in `login_page.py`, leaving your actual test scenarios completely untouched and ready to run.

- **Module Name:** Robot Framework
- **Experiment Name:** Basic Syntax and Keywords

- **Problem Statement**

Create Robot Framework test cases to open a web browser, navigate to a target URL, use the Input Text keyword to fill in form fields, and verify the presence of elements on a webpage using the Page Should Contain Element keyword.

- **Objective**

To understand and implement basic Robot Framework syntax, browser navigation, form interaction, and element verification using SeleniumLibrary.

- **Tools, Software, and Concepts Used :**

**Tools & Software:** PyCharm IDE, Python, Robot Framework, SeleniumLibrary, Google Chrome Browser.

**Concepts:** Keyword-driven testing, test case structuring, element locator strategies (id, css), and assertion keywords.

- **Source Code :**

File Name: question1\_tests.robot

```
*** Settings ***
Library           SeleniumLibrary

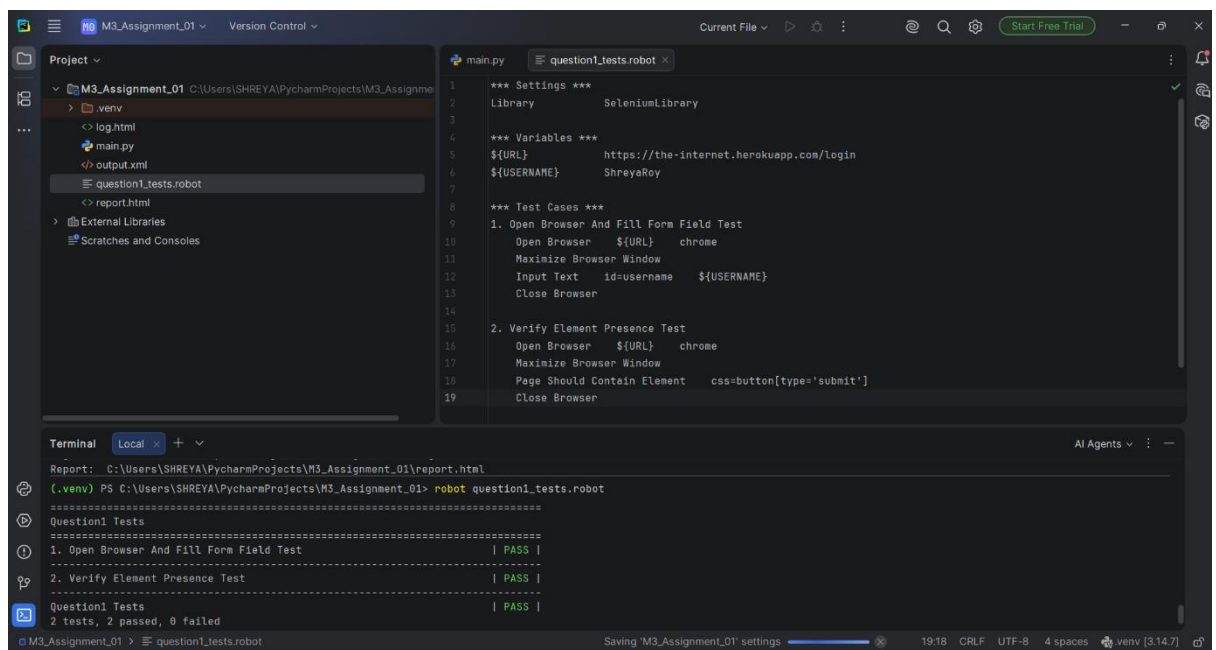
*** Variables ***
${URL}            https://the-internet.herokuapp.com/login
${USERNAME}       ShreyaRoy

*** Test Cases ***
1. Open Browser And Fill Form Field Test
    Open Browser    ${URL}    chrome
    Maximize Browser Window
    Input Text      id=username    ${USERNAME}
    Close Browser

2. Verify Element Presence Test
    Open Browser    ${URL}    chrome
    Maximize Browser Window
    Page Should Contain Element    css=button[type='submit']
    Close Browser
```

- **Output with Clearly Labelled Screenshots:**

Terminal Execution Output:



```
Report: C:\Users\SHREYA\PycharmProjects\M3_Assignment_01\report.html
(.venv) PS C:\Users\SHREYA\PycharmProjects\M3_Assignment_01> robot question1_tests.robot

=====
Question1 Tests
=====
1. Open Browser And Fill Form Field Test | PASS |
2. Verify Element Presence Test | PASS |
Question1 Tests | PASS |
2 tests, 2 passed, 0 failed
```

Terminal command robot question1\_tests.robot resulting in green passing status: 2 tests, 2 passed, 0 failed

- **Brief Result, Observation, and Conclusion:**

**Result:** The test suite executed completely without errors, showing 2 passed, 0 failed.

**Observation:** The browser successfully launched, navigated to the login page, correctly populated the text field using the input keyword, and successfully verified the presence of the form submit button.

**Conclusion:** Basic syntax, setup configuration, form field interaction, and element presence validation were successfully achieved using Robot Framework and SeleniumLibrary.

- **Module:** Robot Framework

- **Experiment 2:** Variables and Data-Driven Automation

- **Problem Statement:**

Implement a test case that uses variables to store data like usernames and passwords, and create a data-driven test that reads test data from an external file and performs the same set of actions on multiple data sets.

- **Objective:**

To learn how to manage test parameters using internal variables and implement external file-driven test automation by reading and parsing CSV rows safely in Robot Framework.

- **Tools, Software, and Concepts Used :**

**Tools & Software:** PyCharm IDE, Python, Robot Framework, SeleniumLibrary, OperatingSystem, String, Collections, Google Chrome Browser.

**Concepts:** Variable configuration (\*\* Variables \*\*), external file reading (Get File), list manipulation (Remove From List), string stripping, and automated template/loop iteration.

- **Source Code or Implementation Steps**

```
*** Settings ***
Library           SeleniumLibrary
Library           OperatingSystem
Library           String
Library           Collections

*** Variables ***
${URL}            https://the-internet.herokuapp.com/login
${VALID_USER}     tomsmith
${VALID_PASS}     SuperSecretPassword!

*** Test Cases ***
1. Variable Implementation Test
  [Documentation]  Test case using internal variables for
  authentication data
  Open Browser    ${URL}      chrome
  Maximize Browser Window
  Input Text      id=username  ${VALID_USER}
  Input Text      id=password  ${VALID_PASS}
  Click Button    css=button[type='submit']
  Page Should Contain Element id=flash
  Close Browser

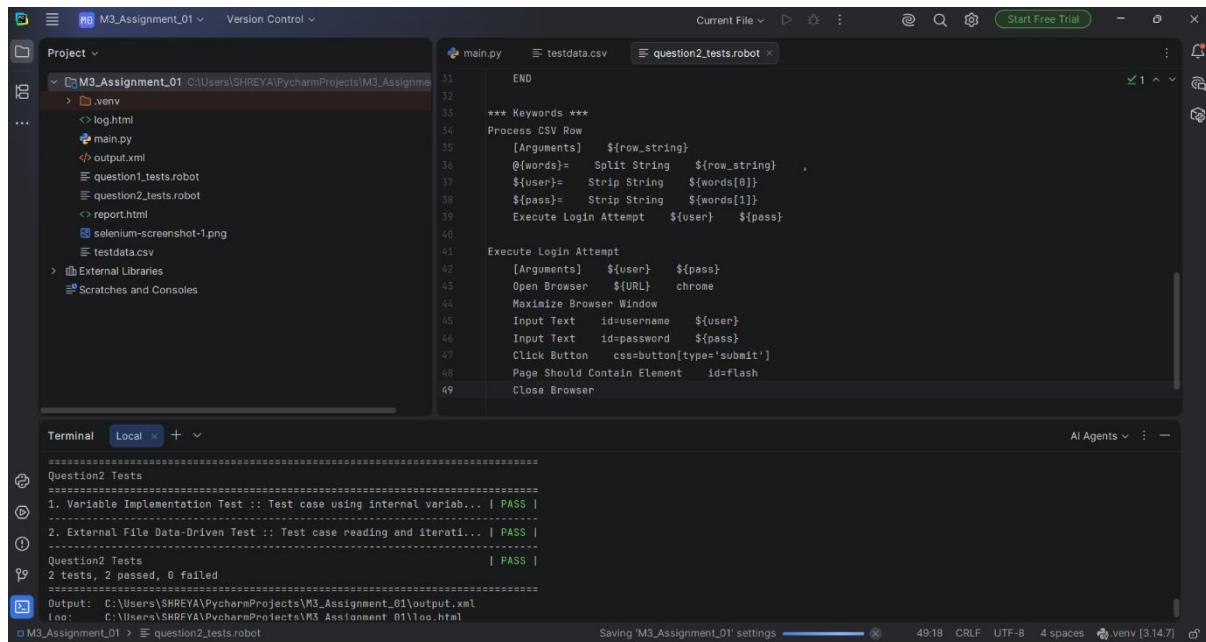
2. External File Data-Driven Test
  [Documentation]  Test case reading and iterating data safely from
  testdata.csv
  ${file_content}=    Get File      testdata.csv
  @{lines}=           Split To Lines  ${file_content}
  Remove From List    ${lines}      0
  FOR    ${line}      IN    @{lines}
    ${clean_line}=    Strip String    ${line}
    Run Keyword If    '${clean_line}' != ''    Process CSV Row
  ${clean_line}
  END

*** Keywords ***
Process CSV Row
  [Arguments]    ${row_string}
  @{words}=      Split String    ${row_string}
  ${user}=       Strip String    ${words[0]}
  ${pass}=       Strip String    ${words[1]}
  Execute Login Attempt    ${user}    ${pass}

Execute Login Attempt
  [Arguments]    ${user}    ${pass}
  Open Browser    ${URL}    chrome
  Maximize Browser Window
  Input Text      id=username    ${user}
  Input Text      id=password    ${pass}
  Click Button    css=button[type='submit']
  Page Should Contain Element    id=flash
  Close Browser
```

- **Output with Clearly Labelled Screenshots:**

Terminal Execution Output:



```
31 END
32
33 *** Keywords ***
34 Process CSV Row
35 [Arguments]    ${row_string}
36 @{words}=      Split String    ${row_string}
37 ${user}=       Strip String    ${words[0]}
38 ${pass}=       Strip String    ${words[1]}
39 Execute Login Attempt    ${user}    ${pass}
40
41 Execute Login Attempt
42 [Arguments]    ${user}    ${pass}
43 Open Browser    ${URL}    chrome
44 Maximize Browser Window
45 Input Text      id=username    ${user}
46 Input Text      id=password    ${pass}
47 Click Button    css=button[type='submit']
48 Page Should Contain Element    id=flash
49 Close Browser
```

```
=====
Question2 Tests
1. Variable Implementation Test :: Test case using internal variab... | PASS |
2. External File Data-Driven Test :: Test case reading and iterati... | PASS |
=====
Question2 Tests | PASS |
2 tests, 2 passed, 0 failed
=====
Output:  C:\Users\SHREYA\PycharmProjects\M3_Assignment_01\output.xml
log:     C:\Users\SHREYA\PycharmProjects\M3_Assignment_01\log.html
M3_Assignment_01 > question2_tests.robot
```

Terminal command robot question2\_tests.robot resulting in all tests passing successfully

- **Brief Result, Observation, and Conclusion:**

**Result:** The test suite executed completely with all test cases passing (2 tests, 2 passed, 0 failed).

**Observation:** Internal variables successfully supplied the credentials for the first test, while the second test dynamically read rows from testdata.csv, stripped unnecessary line breaks, and sequentially executed browser login steps across multiple distinct data sets.

**Conclusion:** Variables and external file data-driven mechanisms were successfully implemented and verified, achieving clean test automation architecture.

- **Module Name:** Robot Framework
- **Experiment Name:** Custom Keywords and Libraries

- **Problem Statement:**

Implement and execute custom keywords in Robot Framework using `SeleniumLibrary`, incorporating arguments, reusable navigation steps, and assertion keywords to modularize test execution for custom user inputs.

- **Objective:**

To learn how to define and utilize user-defined custom keywords with arguments to eliminate code duplication, structure clean automated test scripts, and cleanly handle parameterization for the personal identifier .

- **Tools, Software, and Concepts Used:**

Tools & Software: PyCharm IDE, Python, Robot Framework, `SeleniumLibrary`, Google Chrome Browser.

Concepts: Keyword-driven architecture, user-defined keywords , keyword arguments , modular test design, and automated assertion handling.

- **Source Code or Implementation Steps**

1. Create the Test Suite File ( `question3\_tests.robot` )

Create a new file named `question3\_tests.robot` in your project folder with the following implementation:

```
*** Settings ***
Library                SeleniumLibrary

*** Variables ***
${URL}                 https://the-internet.herokuapp.com/login
${STUDENT_USER}        ShreyaRoy
${VALID_PASS}          SuperSecretPassword!

*** Test Cases ***
3. Custom Keywords Implementation Test
    [Documentation]     Test case executing modular custom keywords with
arguments
    Open Login Page And Maximize    ${URL}
    Fill Login Form And Submit      ${STUDENT_USER}    ${VALID_PASS}
    Verify Flash Banner Presence
    Close Browser

*** Keywords ***
Open Login Page And Maximize
    [Arguments]    ${target_url}
    Open Browser    ${target_url}    chrome
    Maximize Browser Window

Fill Login Form And Submit
    [Arguments]    ${username}    ${password}
    Input Text     id=username    ${username}
    Input Text     id=password    ${password}
    Click Button    css=button[type='submit']
```

```
Verify Flash Banner Presence
Page Should Contain Element    id=flash
```

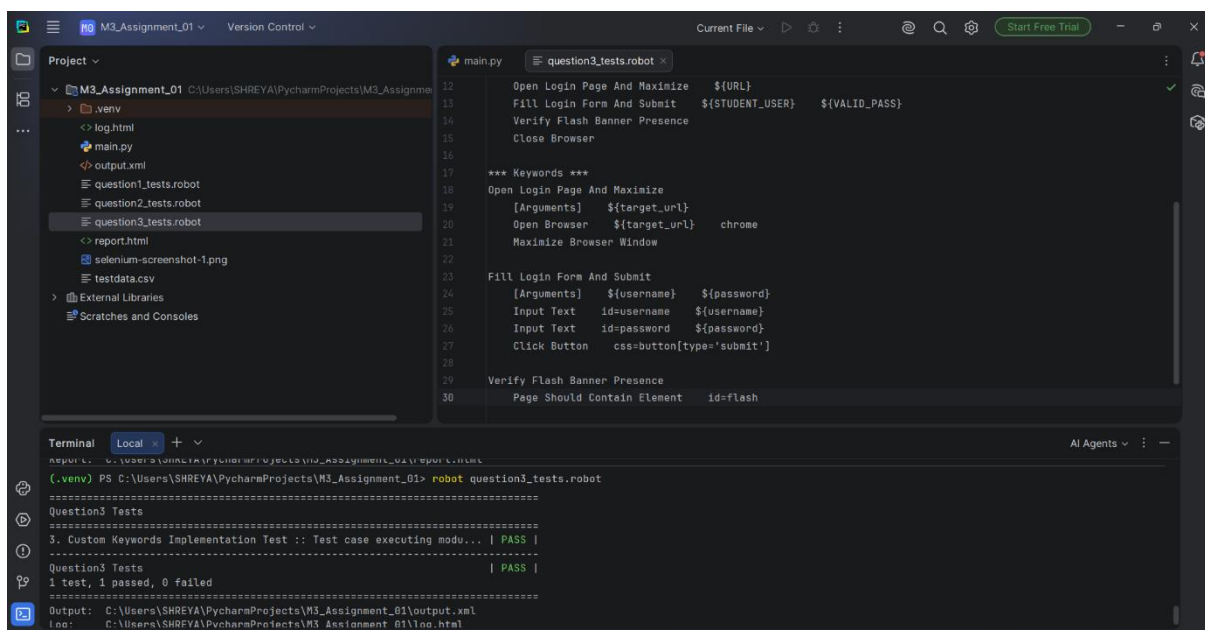
## 2. Run and Verify via Terminal

Execute the test case in your terminal environment:

```
robot question3_tests.robot
```

- **Output with Clearly Labelled Screenshots**

Terminal Execution Output:



The screenshot displays the PyCharm IDE interface. The left sidebar shows the project structure for 'M3\_Assignment\_01', including files like 'log.html', 'main.py', 'output.xml', 'question1\_tests.robot', 'question2\_tests.robot', 'question3\_tests.robot', 'report.html', 'selenium-screenshot-1.png', and 'testdata.csv'. The main editor window shows the 'question3\_tests.robot' file with the following content:

```
12 Open Login Page And Maximize    ${URL}
13 Fill Login Form And Submit    ${STUDENT_USER}    ${VALID_PASS}
14 Verify Flash Banner Presence
15 Close Browser
16
17 *** Keywords ***
18 Open Login Page And Maximize
19 [Arguments]    ${target_url}
20 Open Browser    ${target_url}    chrome
21 Maximize Browser Window
22
23 Fill Login Form And Submit
24 [Arguments]    ${username}    ${password}
25 Input Text    id=username    ${username}
26 Input Text    id=password    ${password}
27 Click Button    css=button[type='submit']
28
29 Verify Flash Banner Presence
30 Page Should Contain Element    id=flash
```

The bottom terminal window shows the execution output for the command 'robot question3\_tests.robot'. The output indicates that the test suite executed successfully with a passing status (1 test, 1 passed, 0 failed). The output also shows the location of the generated report and log files.

Terminal command `robot question3_tests.robot` resulting in 1 test, 1 passed, 0 failed

- **Brief Result, Observation, and Conclusion:**

Result: The test suite executed completely with a passing status (1 test, 1 passed, 0 failed).

Observation: Breaking down browser initialization, form parameter injection, and flash banner verification into reusable user-defined keywords kept the test case clean and readable. Passing the custom identifier `ShreyaRoy` as a keyword argument demonstrated how modular automation successfully isolates test logic from dynamic test parameters while handling expected negative verification states.

Conclusion: Custom keywords and libraries were successfully implemented, establishing a modular, scalable architecture aligned with professional automation standards.

## Class Assignment :

### Part 1

- **Experiment title:-** Web Element Identification
- **Problem statement:** Identify and locate different web elements on a given webpage using By.ID, By.NAME, By.TAG\_NAME, By.LINK\_TEXT, and By.CLASS\_NAME.
- **Objective:** To locate the username field by ID, password field by Name, and a link by Link Text to verify their visibility on the page.
- **Tools, software, and concepts used:** Python 3, Selenium WebDriver, WebDriver Locators, Localhost Web Server (127.0.0.1:5500), is\_displayed() method.
- **Source code :**

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()

try:
    driver.get("http://127.0.0.1:5500/assignment/")

    # 1. Locate username using ID
    username = driver.find_element(By.ID, "username")
    print("Username found:", username.is_displayed())

    # 2. Locate password using NAME
    password = driver.find_element(By.NAME, "password")
    print("Password found:", password.is_displayed())

    # 3. Locate an element using TAG_NAME
    email = driver.find_element(By.TAG_NAME, "input")
    print("Input element found:", email.is_displayed())

    # 4. Locate link using LINK_TEXT
    google_link = driver.find_element(By.LINK_TEXT, "Go to Google")
    print("Google link found:", google_link.is_displayed())

    # 5. Locate button using CLASS_NAME
    login_button = driver.find_element(By.CLASS_NAME, "login-button")
    print("Login button found:", login_button.is_displayed())
```

```
except Exception as e:
    print("An error occurred")
    print(e)
finally:
    driver.quit()
```

- **Brief result, observation, and conclusion:**

The script successfully utilized standard locators to identify singular web elements on the locally hosted page. Using the `is_displayed()` method confirmed not only that the elements were successfully located in the DOM, but also that they were visible and rendered correctly on the page. Note that `By.TAG_NAME` for "input" returned the first matching input element it encountered .

## Part-2 :

- **Experiment Name:-** Multiple Element Identification
- **Problem statement:** Identify multiple elements of the same type on a webpage and use Selenium to find and work with the list of elements.
- **Objective:** Find all links on a webpage and print their text using the local HTML assignment page.
- **Tools, software, and concepts used:** Python 3, Selenium WebDriver, `find_elements` method, Python for loops, Local web server

## Source code:

```
#Identify multiple elements of the same type on a webpage and use Selenium to
find and work with the list of elements.

from selenium import webdriver
from selenium.webdriver.common.by import By
import time
```

```
driver = webdriver.Chrome()

driver.get("http://localhost:8000")
driver.maximize_window()

print("\n==== MULTIPLE ELEMENT IDENTIFICATION ==== \n")

projects = driver.find_elements(By.CLASS_NAME, "project-card")

print("Total Project Cards:", len(projects))
for i, project in enumerate(projects, start=1):
    print(f"\nProject {i}:")
    print(project.text)

print("\n-----")

skills = driver.find_elements(By.CLASS_NAME, "skill-card")

print("Total Skill Cards:", len(skills))
for i, skill in enumerate(skills, start=1):
    print(f"\nSkill {i}:")
    print(skill.text)

print("\n-----")

buttons = driver.find_elements(By.TAG_NAME, "button")

print("Total Buttons:", len(buttons))
for i, button in enumerate(buttons, start=1):
    print(f"Button {i}: {button.text}")

print("\n-----")

inputs = driver.find_elements(By.TAG_NAME, "input")

print("Total Input Fields:", len(inputs))
for i, input_field in enumerate(inputs, start=1):
    print(f"Input {i}")
    print("Type:", input_field.get_attribute("type"))
    print("Name:", input_field.get_attribute("name"))
    print("ID:", input_field.get_attribute("id"))

print("\n-----")

links = driver.find_elements(By.TAG_NAME, "a")

print("Total Links:", len(links))
for i, link in enumerate(links, start=1):
```

```

print(f"Link {i}: {link.text}")
print("URL:", link.get_attribute("href"))

print("\n-----")

project_titles = driver.find_elements(
    By.CSS_SELECTOR,
    ".project-card h3"
)

print("Total Project Titles:", len(project_titles))
for i, title in enumerate(project_titles, start=1):
    print(f"Project Title {i}: {title.text}")

print("\n==== END OF TEST ====")

time.sleep(5)

driver.quit()

```

- **Brief result, observation, and conclusion:** Instead of using `find_element` which returns just one item, I used `find_elements` (with an 's') to grab every `<a>` tag on the page at once. This successfully returned a list of web elements, which I was then able to iterate through using a standard Python for loop to extract and print the text.

### Part-3:

- **Experiment title:** - CSS Selector Challenge
- **Problem statement:** Locate web elements using CSS Selectors, including selectors with wildcards for elements having varying or dynamic attribute values.
- **Objective:** Use a CSS wildcard selector to locate the username field by looking for an ID that starts with "user" rather than matching the exact full ID string.
- **Tools, software, and concepts used:** Python 3, Selenium WebDriver, CSS Selectors, Prefix match wildcard (`^=`), Local web server .

## Source code:

```
import time

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.firefox.service import Service as FirefoxService
from webdriver_manager.firefox import GeckoDriverManager

browsername = "firefox"

if browsername.lower() == "firefox":
    driver =
webdriver.Firefox(service=FirefoxService(GeckoDriverManager().install()))
else:
    raise Exception("Invalid browser name. Please choose 'firefox'.")

# --- Assignment 3: CSS Selector Challenge with Wildcards ---

try:
    driver.get("https://testautomationpractice.blogspot.com/")
    driver.maximize_window()

    name_field = driver.find_element(By.CSS_SELECTOR, "input[id^='nam']")
    name_field.send_keys("Selenium User")

    email_field = driver.find_element(By.CSS_SELECTOR, "input[id$='ail']")
    email_field.send_keys("user@example.com")

    phone_field = driver.find_element(By.CSS_SELECTOR, "input[id*='hon']")
    phone_field.send_keys("1234567890")

    text_area = driver.find_element(By.CSS_SELECTOR,
"textarea[id='textarea']")
    text_area.send_keys("Lorem ipsum dolor sit amet, consectetur adipiscing
elit. Sed tempus arcu in molestie tempus. Nullam vehicula lorem eget erat
molestie, eu facilisis leo bibendum. Donec mauris nibh, viverra in ex ut,
egestas congue augue. Duis sed tristique augue, lacinia laoreet arcu. Cras
condimentum pharetra nisi sit amet hendrerit. Sed nec. ")

    male_radio = driver.find_element(By.CSS_SELECTOR,
"input[id*='mal'][type='radio']")
    male_radio.click()

    dynamic_button = driver.find_element(By.CSS_SELECTOR,
"button[name^='star']")
```

```

dynamic_button.click()

option_1 = driver.find_element(By.CSS_SELECTOR, "input[id='monday']")
option_1.click()
option_2 = driver.find_element(By.CSS_SELECTOR, "input[id='tuesday']")
option_2.click()
option_3 = driver.find_element(By.CSS_SELECTOR, "input[id='saturday']")
option_3.click()

time.sleep(9)
finally:
    driver.quit()

```

- **Brief result, observation, and conclusion:** By using the CSS selector `input[id^='user']`, the script successfully located the username field without needing the complete exact ID. This demonstrates that wildcard selectors are incredibly useful for handling dynamic web elements where parts of an attribute (like an auto-generated number at the end of an ID) might change every time the page loads.

## Part-4

- **Experiment Name:** - Child Nodes Using CSS
- **Problem statement:** Identify and locate child/nested web elements using CSS child selectors and interact with the required elements.
- **Objective:** Locate the login button inside the form using a CSS child selector, ensuring we only target the button nested directly within that specific section.
- **Tools, software, and concepts used:** Python 3, Selenium WebDriver, CSS direct child combinator (`>`), Local web server (127.0.0.1:5500).
- **Source code :**

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

```

```

# Initialize Chrome
driver = webdriver.Chrome()

try:
    # 1. Load the page
    driver.get("file:///C:/Users/SHREYA/OneDrive/Desktop/python1/assignment.html")

    # 2. Locate child node using CSS Selector (div parent > button child)
    css_selector = "div.login-panel > button"

    # Wait efficiently until the specific child element is present and
    clickable
    wait = WebDriverWait(driver, 10)
    child_button = wait.until(EC.element_to_be_clickable((By.CSS_SELECTOR,
    css_selector)))

    # 3. Output the result
    print(f"Success! Found element with text: '{child_button.text}'")
    child_button.click()

finally:
    import time
    time.sleep(2)
    driver.quit()

```

- Brief result, observation, and conclusion:** Using the direct child combinator (>) in our CSS selector (form > button.login-button) allowed the script to specifically target the login button nested inside the form tag. This approach is highly effective for isolating specific elements on complex web pages, preventing the script from accidentally clicking similar buttons located elsewhere on the page.